INSTITUTO NTA
Formação em Engenharia de Dados

Python para Engenharia de Dados

Parte 2: Decisão, laços e funções

APOSTILA DO ALUNO | PARTE 2 DE 2

Conteúdo

- Retomando a Parte 1
- Decisão: if, elif e else
- Laço for, range e enumerate
- Laço while
- List comprehension
- Funções
- Mini projeto: validação de vendas e relatório por cliente

Responsável técnico: Thiago Azeredo Rodrigues

Sumário

1. Retomando a Parte 1 . . . 3
2. Decisão: if, elif e else . . . 4
3. Laço for . . . 7
4. Laço while . . . 12
5. List comprehension . . . 14
6. Funções . . . 16
7. Mini projeto: validação de vendas e relatório por cliente . . . 19
8. Erros comuns em decisões, laços e funções . . . 23
9. Exercícios . . . 23
10. Resumo e próximos passos . . . 24
Gabarito dos exercícios . . . 25
Glossário . . . 32

Como usar esta apostila

Esta apostila acompanha a **Parte 2** do curso Python para Engenharia de Dados. Ela é o material de estudo: traz a teoria completa, exemplos com o resultado de cada código, exercícios e gabarito. Na aula, o foco é executar o código ao vivo. Aqui você revisa e aprofunda no seu ritmo.

Material	Para quê
Apostila (este PDF)	Estudo e consulta: teoria, exemplos com saída, exercícios e gabarito
Notebook `.ipynb`	Prática: todo o código da apostila pronto para executar e alterar

> **Antes de começar.** Esta parte pressupõe a Parte 1: variáveis, tipos de dados, operadores e coleções. Você precisa do ambiente montado lá (Python 3.11 ou superior e VS Code). Alternativa sem instalar nada: abra o notebook no Google Colab.

1. Retomando a Parte 1

Na Parte 1 você montou a base: o que é programar, o ambiente (VS Code), as características do Python, as variáveis, os tipos de dados, os operadores e as quatro coleções. Nesta Parte 2 vamos fazer o código **tomar decisões**, **repetir tarefas** e **ficar organizado em blocos reutilizáveis**. É aqui que o Python deixa de ser uma calculadora e passa a processar dados de verdade.

Conceito da Parte 1	Em uma frase
Variável	Nome que referencia um valor (`total = 0`)
Tipos básicos	`int`, `float`, `str`, `bool` e `None`
Operadores	Aritméticos, de comparação (`==`, `>`), lógicos (`and`, `or`, `not`) e de pertinência (`in`)
`list`	Sequência ordenada e mutável: `[1, 2, 3]`
`tuple`	Sequência ordenada e imutável: `(1, 2)`
`dict`	Pares chave e valor: `{"id": 1}`
`set`	Valores únicos, sem ordem: `{1, 2, 3}`
`try` e `except`	Captura um erro sem interromper o programa

Durante toda a aula usaremos a mesma tabela de vendas do mini projeto da Parte 1: uma **lista de dicionários**, em que cada dicionário é uma venda. **Execute a célula abaixo primeiro**, porque quase todos os exemplos dependem dela.

```
vendas = [
    {"id": 1, "cliente": "Ana",   "produto": "Notebook", "valor": 4500.00},
    {"id": 2, "cliente": "Bruno", "produto": "Mouse",    "valor": 89.90},
    {"id": 3, "cliente": "Ana",   "produto": "Teclado",  "valor": 250.00},
    {"id": 4, "cliente": "Carla", "produto": "Monitor",  "valor": 1200.00},
    {"id": 5, "cliente": "Bruno", "produto": "Notebook", "valor": 5200.00},
    {"id": 6, "cliente": "Diego", "produto": "Mouse",    "valor": 79.90},
]
```

Uma conferência rápida: quantas vendas temos e como é a primeira.

```
print(len(vendas))
print(vendas[0])
```

SAÍDA

```
6
{'id': 1, 'cliente': 'Ana', 'produto': 'Notebook', 'valor': 4500.0}
```

Dica de notebook. Se você reiniciar o kernel ou abrir o notebook do zero, execute a célula da lista `vendas` antes de qualquer outra. Sem ela aparece `NameError`.

2. Decisão: if, elif e else

Até agora o Python executava todas as linhas, sempre, de cima para baixo. Programas reais precisam **escolher caminhos**: "se a venda é grande, faça isto; senão, faça aquilo". Isso se chama **decisão** (ou *condicional*). É o mesmo raciocínio do dia a dia: se o semáforo está vermelho, pare; senão, siga.

2.1 O if

A palavra `if` significa "se". Depois dela vem uma **condição**: qualquer expressão que resulte em `True` ou `False`, como as comparações do Módulo 7 da Parte 1. O bloco recuado logo abaixo **só executa se a condição for verdadeira**.

```
if condicao:
    # bloco executado só se a condição for True
```

- O **dois-pontos** no fim da linha do `if` é obrigatório.
- O **recuo** (4 espaços) das linhas seguintes define o que está dentro do `if`.

```
valor = 120

if valor > 100:
    print("Venda grande")
```

SAÍDA

```
Venda grande
```

Agora com o valor 80. A condição é falsa, então o bloco **não executa**. Já a última linha, sem recuo, não pertence ao `if` e executa sempre:

```
valor = 80

if valor > 100:
    print("Venda grande")
print("Verificação concluída")
```

SAÍDA

```
Verificação concluída
```

2.2 O else

O `else` significa "senão". Define o que fazer quando a condição é falsa. Só um dos dois blocos executa, nunca os dois.

```
valor = 80

if valor > 100:
    print("Venda grande")
else:
    print("Venda pequena")
```

SAÍDA

```
Venda pequena
```

2.3 O elif

E se existirem **mais de dois caminhos**? Por exemplo: venda grande, média ou pequena. Para isso existe o `elif`, que é a junção de "senão, se". O Python testa as condições **de cima para baixo**, executa o bloco da **primeira** que for verdadeira e ignora o resto.

```
valor = 250

if valor >= 1000:
    print("Venda grande")
elif valor >= 100:
    print("Venda média")
else:
    print("Venda pequena")
```

SAÍDA

```
Venda média
```

Como o Python para na primeira condição verdadeira, **a ordem importa**. Veja o que acontece se invertermos as duas primeiras condições com um valor de 4500:

```
valor = 4500

if valor >= 100:
    print("Venda média")
elif valor >= 1000:
    print("Venda grande")
else:
    print("Venda pequena")
```

SAÍDA

```
Venda média
```

Um valor de 4500 foi classificado como média, porque 4500 já é maior ou igual a 100 e o Python nunca chegou à segunda condição. Regra prática: **comece pela condição mais restritiva**.

2.4 Condições compostas

Os operadores lógicos do Módulo 7 da Parte 1 permitem combinar condições. `and` exige que **as duas** sejam verdadeiras:

```
cliente = "Ana"
valor = 4500.00

if cliente == "Ana" and valor > 1000:
    print("Venda grande da Ana")
```

SAÍDA

```
Venda grande da Ana
```

`or` exige que **pelo menos uma** seja verdadeira. E `in` pergunta se um valor está numa coleção, o que evita repetir vários `or`:

```
produto = "Mouse"

if produto == "Mouse" or produto == "Teclado":
    print("Periférico (com or)")

if produto in ["Mouse", "Teclado", "Fone"]:
    print("Periférico (com in)")
```

SAÍDA

```
Periférico (com or)
Periférico (com in)
```

`not` inverte a condição:

```
ativo = False

if not ativo:
    print("Cliente inativo")
```

SAÍDA

```
Cliente inativo
```

2.5 Verificando valores vazios e nulos

Lembra dos valores “verdadeiros” e “falsos” da Parte 1? Eles aparecem muito em decisões. Uma lista vazia conta como falsa, e uma lista com itens conta como verdadeira:

```
itens = []

if itens:
    print("Há itens")
else:
    print("Lista vazia")
```

SAÍDA

```
Lista vazia
```

Para testar o **nulo** (`None`, o `NULL` do SQL), usamos `is None`:

```
telefone = None

if telefone is None:
    print("Telefone não informado")
```

SAÍDA

```
Telefone não informado
```

Atenção. `if not valor` é verdadeiro tanto para `None` quanto para `0`. Se o valor zero é legítimo nos seus dados (um saldo zerado, por exemplo), use `is None` para testar só o nulo. Confundir os dois é um bug clássico em tratamento de dados.

2.6 Decisões dentro de decisões

Um `if` pode ficar dentro de outro. Cada nível ganha mais um recuo de 4 espaços.

```
valor = 1500
cliente = "Ana"

if valor > 1000:
    if cliente == "Ana":
        print("Venda grande de cliente conhecido")
    else:
        print("Venda grande")
```

SAÍDA

```
Venda grande de cliente conhecido
```

Muitas vezes um `and` deixa o código mais enxuto do que dois `if` aninhados. Use o que ficar mais fácil de ler.

2.7 Decisão em uma linha

Quando a decisão serve só para escolher **entre dois valores**, existe uma forma curta, chamada *expressão condicional*: `valor_se_verdadeiro if condição else valor_se_falso`. Ela será útil quando chegarmos às comprehensions.

```
valor = 1500
faixa = "grande" if valor >= 1000 else "pequena"
print(faixa)
```

SAÍDA

```
grande
```

3. Laço for

Dados chegam em grande quantidade: mil linhas, um milhão de registros. Ninguém escreve mil vezes o mesmo comando. Um **laço** (em inglês, *loop*) manda o Python **repetir** um bloco de código.

Uma analogia: imagine uma pilha de caixas e a ordem “para cada caixa da pilha, abra, confira o conteúdo e anote”. Você escreve a instrução uma vez, e ela vale para todas as caixas. Python tem dois laços: o **for**, que repete **para cada item** de uma coleção, e o **while**, que repete **enquanto uma condição for verdadeira** (veremos no Módulo 4).

3.1 A estrutura do for

```
for item in colecao:
    # tudo que está recuado (4 espaços) é repetido a cada volta
```

- `item` é um nome que **você escolhe**. A cada volta, ele passa a valer o próximo valor da coleção.
- `colecao` é o que será percorrido: uma lista, um texto, um dicionário, um `range`...
- O **dois-pontos** no fim da linha e o **recuo** das linhas seguintes são obrigatórios. O recuo diz o que está dentro do laço.

Com uma lista de três cidades, o `print` está recuado e executa a cada volta:

```
cidades = ["BH", "SP", "RJ"]

for cidade in cidades:
    print("Cidade:", cidade)
```

SAÍDA

```
Cidade: BH
Cidade: SP
Cidade: RJ
```

Volta a volta: na primeira, `cidade` valia `"BH"` e imprimiu; na segunda, valia `"SP"`; na terceira, `"RJ"`. Acabaram os itens, e o laço terminou.

Uma linha **sem recuo** logo depois do laço não faz parte dele: executa uma única vez, quando o laço termina.

```
for cidade in cidades:
    print("Dentro do laço:", cidade)
print("Fora do laço: terminou")
```

SAÍDA

```
Dentro do laço: BH
Dentro do laço: SP
Dentro do laço: RJ
Fora do laço: terminou
```

Um texto também é uma sequência, então o `for` o percorre letra a letra:

```
for letra in "Dados":
    print(letra)
```

SAÍDA

```
D
a
d
o
s
```

3.2 A função range

Às vezes não temos uma coleção pronta, só queremos repetir algo um número de vezes. A função `range()` fabrica uma sequência de números inteiros:

- `range(5)` gera 0, 1, 2, 3, 4 (começa em 0 e **não inclui** o 5).
- `range(1, 6)` gera 1, 2, 3, 4, 5 (o início entra, o fim não, como no fatiamento).
- `range(0, 10, 2)` gera 0, 2, 4, 6, 8 (o terceiro número é o **passo**).

Para ver os números de uma vez, envolvemos em `list()`:

```
print(list(range(5)))
print(list(range(1, 6)))
print(list(range(0, 10, 2)))
```

SAÍDA

```
[0, 1, 2, 3, 4]
[1, 2, 3, 4, 5]
[0, 2, 4, 6, 8]
```

```
for i in range(1, 4):
    print("Volta número", i)
```

SAÍDA

```
Volta número 1
Volta número 2
Volta número 3
```

3.3 Acumuladores: somando e contando

O padrão mais usado em dados é o **acumulador**: uma variável criada **antes** do laço, que vai sendo atualizada a cada volta. Vamos somar uma lista de valores na mão, como a função `sum()` faz por baixo dos panos.

```
valores = [50, 120, 300, 80, 150]

total = 0
for v in valores:
    total = total + v

print(total)
```

SAÍDA

```
700
```

Acompanhe o que acontece com `total` em cada volta:

```
início:   total = 0
volta 1:  v = 50    total = 0 + 50    = 50
volta 2:  v = 120   total = 50 + 120  = 170
volta 3:  v = 300   total = 170 + 300 = 470
volta 4:  v = 80    total = 470 + 80  = 550
volta 5:  v = 150   total = 550 + 150 = 700
```

Lembra do atalho `+=`? `total += v` faz o mesmo que `total = total + v`. E por que o `total = 0` fica **fora** do laço? Porque, se estivesse dentro, voltaria a zero em toda volta e perderíamos a soma.

> **Atenção.** Esquecer de criar o acumulador antes do laço gera `NameError`. Colocá-lo dentro do laço faz o valor zerar a cada volta. São os dois erros mais comuns de quem está começando.

3.4 Laço com decisão

Juntando o Módulo 2 com este: somar **apenas** as vendas acima de 100 e contar quantas são. Observe os **dois níveis de recuo**: o `if` está dentro do `for`, e as duas linhas de baixo estão dentro do `if`.

```
total_grandes = 0
qtd_grandes = 0

for v in valores:
    if v > 100:
        total_grandes += v
        qtd_grandes += 1

print("Total:", total_grandes)
print("Quantidade:", qtd_grandes)
```

SAÍDA

```
Total: 570
Quantidade: 3
```

Este é o código **imperativo** que vimos no Módulo 4.4 da Parte 1: uma receita passo a passo. Agora você consegue ler cada linha dele. O resultado, 570, é o mesmo da versão declarativa.

3.5 Percorrendo dicionários

Quando o `for` percorre um dicionário diretamente, recebe uma **chave** por volta:

```
cliente = {"id": 1, "nome": "Ana", "cidade": "Belo Horizonte"}

for chave in cliente:
    print(chave)
```

SAÍDA

```
id
nome
cidade
```

Para receber **chave e valor juntos**, usamos `.items()`. O `for` agora tem dois nomes, separados por vírgula: o primeiro recebe a chave e o segundo, o valor (é o desempacotamento de tupla da Parte 1).

```
for chave, valor in cliente.items():
    print(chave, "=", valor)
```

SAÍDA

```
id = 1
nome = Ana
cidade = Belo Horizonte
```

O caso mais importante em dados é a **lista de dicionários**, que é uma tabela. Cada volta entrega uma linha da tabela, e acessamos as colunas pelas chaves:

```
for venda in vendas:
    print(venda["id"], venda["cliente"], venda["valor"])
```

SAÍDA

```
1 Ana 4500.0
2 Bruno 89.9
3 Ana 250.0
4 Carla 1200.0
5 Bruno 5200.0
6 Diego 79.9
```

3.6 O enumerate: laço com posição

Às vezes precisamos, além do item, da **posição** dele: numerar linhas, apontar em qual linha do arquivo está um erro. A função `enumerate()` entrega pares (posição, item), e o `for` desempacota cada par em dois nomes. A contagem começa em 0:

```
produtos = ["Mouse", "Teclado", "Monitor"]

for posicao, produto in enumerate(produtos):
    print(posicao, produto)
```

SAÍDA

```
0 Mouse
1 Teclado
2 Monitor
```

Com `start=1`, a contagem começa onde quisermos. Útil para numerar listas para humanos:

```
for posicao, produto in enumerate(produtos, start=1):
    print(f"{posicao}. {produto}")
```

SAÍDA

```
1. Mouse
2. Teclado
3. Monitor
```

Na engenharia de dados. Ao validar um arquivo, `enumerate` permite dizer “erro na linha 37” em vez de apenas “há um erro”. Quem recebe o relatório agradece.

3.7 break e continue

Duas palavras controlam o andamento de um laço, e as duas costumam aparecer dentro de um `if`:

- **break**: **sai** do laço imediatamente, sem terminar as voltas que faltam.
- **continue**: **pula** o restante desta volta e vai direto para a próxima.

Com `break`, paramos assim que achamos a primeira venda acima de 200:

```
for v in valores:
    if v > 200:
        print("Achei uma venda acima de 200:", v)
        break
```

SAÍDA

```
Achei uma venda acima de 200: 300
```

Com `continue`, ignoramos as vendas abaixo de 100 e tratamos só as outras:

```
for v in valores:
    if v < 100:
        continue
    print("Venda considerada:", v)
```

SAÍDA

```
Venda considerada: 120
Venda considerada: 300
Venda considerada: 150
```

3.8 Laços dentro de laços

Um `for` pode ficar dentro de outro. A cada volta do laço **externo**, o laço **interno** roda todas as suas voltas. Com 2 clientes e 2 produtos, o `print` executa 2 x 2 = 4 vezes:

```
for cliente in ["Ana", "Bruno"]:
    for produto in ["Mouse", "Teclado"]:
        print(cliente, "-", produto)
```

SAÍDA

```
Ana - Mouse
Ana - Teclado
Bruno - Mouse
Bruno - Teclado
```

Um caso real: para cada cliente, listar suas compras na tabela de vendas.

```
for cliente in ["Ana", "Bruno"]:
    print("Compras de", cliente)
    for venda in vendas:
        if venda["cliente"] == cliente:
            print("   ", venda["produto"], venda["valor"])
```

SAÍDA

```
Compras de Ana
    Notebook 4500.0
    Teclado 250.0
Compras de Bruno
    Mouse 89.9
    Notebook 5200.0
```

Atenção. Funciona, mas percorre a tabela inteira para cada cliente. Com milhões de linhas isso fica lento. No mini projeto (Módulo 7) resolvemos o mesmo problema em uma única passada, usando um dicionário.

4. Laço while

O `while` repete **enquanto a condição for verdadeira**. A cada volta, o Python testa a condição de novo, e quando ela fica falsa o laço termina. Use `while` quando você **não sabe quantas voltas** serão necessárias. Primeiro, uma contagem regressiva:

```
contagem = 3

while contagem > 0:
    print("Faltam", contagem)
    contagem -= 1

print("Pronto!")
```

SAÍDA

```
Faltam 3
Faltam 2
Faltam 1
Pronto!
```

Observe a linha `contagem -= 1`: é ela que, a cada volta, aproxima a condição de ficar falsa. Sem ela, `contagem` seria sempre 3 e o laço nunca terminaria.

Atenção. Um laço que nunca termina se chama **laço infinito**. Se acontecer no notebook, clique no botão de parar (*Interrupt*) da célula. Sempre confira se algo dentro do laço muda a condição.

4.1 Nova tentativa (retry)

Um exemplo clássico de engenharia de dados: chamar um serviço e, se falhar, tentar de novo, com um limite de tentativas. Aqui simulamos que o serviço só responde na segunda tentativa. A condição combina dois testes: continue **enquanto não** deu certo **e** ainda restam tentativas.

```
tentativa = 0
sucesso = False

while not sucesso and tentativa < 3:
    tentativa += 1
    print("Tentativa", tentativa)
    if tentativa == 2:
        sucesso = True

print("Deu certo?", sucesso)
```

SAÍDA

```
Tentativa 1
Tentativa 2
Deu certo? True
```

4.2 Processando uma fila

Outro uso comum: processar uma fila de tarefas até ela esvaziar. Lembre que uma lista vazia conta como falsa, então `while fila:` significa “enquanto a fila tiver itens”. O método `.pop(0)` **remove e devolve** o primeiro item da lista.

```
fila = ["arquivo_a.csv", "arquivo_b.csv", "arquivo_c.csv"]

while fila:
    arquivo = fila.pop(0)
    print("Processando", arquivo, "| restam", len(fila))

print("Fila vazia")
```

SAÍDA

```
Processando arquivo_a.csv | restam 2
Processando arquivo_b.csv | restam 1
Processando arquivo_c.csv | restam 0
Fila vazia
```

4.3 for ou while?

Situação	Use
Percorrer uma lista, um dicionário, um texto	`for`
Repetir um número conhecido de vezes	`for` com `range`
Repetir até uma condição mudar, sem saber quantas voltas	`while`
Nova tentativa até dar certo, com limite	`while`
Esvaziar uma fila	`while`

5. List comprehension

Um padrão se repete o tempo todo: criar uma lista vazia, percorrer algo com `for` e ir fazendo `append`. A **comprehension** é um atalho do Python para esse padrão, escrito em uma única linha. Vamos primeiro ver o jeito longo, que você já domina, para entender o atalho.

5.1 Do laço para a comprehension

Objetivo: obter uma lista só com os valores de cada venda. Pelo jeito longo:

```
valores_laco = []
for v in vendas:
    valores_laco.append(v["valor"])

print(valores_laco)
```

SAÍDA

```
[4500.0, 89.9, 250.0, 1200.0, 5200.0, 79.9]
```

Pelo atalho, tudo cabe numa linha entre colchetes:

```
valores_vendas = [v["valor"] for v in vendas]

print(valores_vendas)
```

SAÍDA

```
[4500.0, 89.9, 250.0, 1200.0, 5200.0, 79.9]
```

A leitura é a mesma do laço, só que **o resultado vem primeiro**. Compare as peças:

Peça	No laço	Na comprehension
O que guardar	`valores_laco.append(v["valor"])`	`v["valor"]` (no início)
De onde vem	`for v in vendas:`	`for v in vendas` (no meio)
Onde fica a lista	`valores_laco = []` antes do laço	Os próprios colchetes `[ ]`

Confira que as duas listas são iguais:

```
print(valores_vendas == valores_laco)
```

SAÍDA

```
True
```

5.2 Transformando os itens

A expressão do início pode ser qualquer cálculo. Por exemplo, os valores com 10% de desconto, arredondados em 2 casas:

```
com_desconto = [round(v["valor"] * 0.9, 2) for v in vendas]

print(com_desconto)
```

SAÍDA

```
[4050.0, 80.91, 225.0, 1080.0, 4680.0, 71.91]
```

5.3 Filtrando com if

Um `if` **no fim** da comprehension filtra: só entram os itens para os quais a condição é verdadeira.

```
acima_de_100 = [v["valor"] for v in vendas if v["valor"] > 100]

print(acima_de_100)
```

SAÍDA

```
[4500.0, 250.0, 1200.0, 5200.0]
```

Agora você entende a linha declarativa do Módulo 4.4 da Parte 1. Era uma comprehension dentro da função `sum()`, sem os colchetes. Usamos de novo a lista `valores = [50, 120, 300, 80, 150]` do Módulo 3.3, e o resultado é o mesmo 570:

```
print(sum(v for v in valores if v > 100))
```

SAÍDA

```
570
```

5.4 if e else dentro da comprehension

Para **escolher entre dois valores** (a expressão condicional do Módulo 2.7), o `if ... else` vai **antes** do `for`. A regra para não errar: o `if` sozinho, que filtra, fica **no fim**; o `if ... else`, que escolhe, fica **no início**.

```
faixas = ["grande" if v["valor"] >= 1000 else "pequena" for v in vendas]

print(faixas)
```

SAÍDA

```
['grande', 'pequena', 'pequena', 'grande', 'grande', 'pequena']
```

Com apenas duas faixas, a venda de 250 fica como pequena. Para três faixas, usaríamos a função `classificar_venda`, que criamos no Módulo 6.

5.5 Comprehension de set e de dict

Com chaves em vez de colchetes, o mesmo atalho cria um `set` (valores únicos) ou um `dict`. Para o `dict`, escrevemos `chave: valor` no início.

```
clientes = {v["cliente"] for v in vendas}
print(sorted(clientes))
```

SAÍDA

```
['Ana', 'Bruno', 'Carla', 'Diego']
```

```
valor_por_id = {v["id"]: v["valor"] for v in vendas}
print(valor_por_id)
```

SAÍDA

```
{1: 4500.0, 2: 89.9, 3: 250.0, 4: 1200.0, 5: 5200.0, 6: 79.9}
```

5.6 Quando não usar

A comprehension é para **montar uma coleção** de forma curta. Se você precisa de várias linhas de lógica dentro do laço, de `break`, `continue` ou de efeitos como imprimir, use o `for` normal. Se a comprehension ficou difícil de ler, ela não está ajudando.

> **Na engenharia de dados.** Comprehensions são ótimas para estruturas pequenas: extrair colunas de uma resposta de API, filtrar uma lista de arquivos, montar um dicionário de parâmetros. Para tabelas grandes, o caminho é pandas ou Spark.

6. Funções

Até aqui, se precisarmos repetir uma lógica em dois lugares, copiamos o código. Isso gera erros: corrigimos num lugar e esquecemos do outro. Uma **função** é um bloco de código **com nome**, que podemos executar quantas vezes quisermos. Você já usa funções prontas: `print`, `len`, `sum`, `sorted`. Agora vai criar as suas.

Uma analogia: um liquidificador. Entram ingredientes (os **parâmetros**), acontece um processo e sai uma vitamina (o **retorno**). Você não precisa refazer o liquidificador a cada vitamina.

6.1 Criando e chamando uma função

A palavra `def` (de *define*) cria a função. Depois vêm o nome, parênteses, dois-pontos e o bloco recuado. **Criar** a função não executa nada, apenas ensina o Python a fazer aquilo. Por isso a célula abaixo não mostra saída.

```
def saudacao():
    print("Olá, turma de engenharia de dados!")
```

Para **executar**, chamamos a função pelo nome, com parênteses:

```
saudacao()
saudacao()
```

SAÍDA

```
Olá, turma de engenharia de dados!
Olá, turma de engenharia de dados!
```

6.2 Parâmetros

Os **parâmetros** ficam entre os parênteses e são as entradas da função. Ao chamar, os valores passados (os **argumentos**) são ligados aos parâmetros.

```
def saudacao_nome(nome):
    print(f"Olá, {nome}!")

saudacao_nome("Ana")
saudacao_nome("Bruno")
```

SAÍDA

```
Olá, Ana!
Olá, Bruno!
```

6.3 Retorno com return

Muitas funções devem **devolver um resultado** para ser usado depois. Isso é o `return`. Assim que o `return` executa, a função termina.

```
def media(numeros):
    return sum(numeros) / len(numeros)

resultado = media([7.5, 8.0, 6.5])
print(round(resultado, 2))
```

SAÍDA

```
7.33
```

Atenção a uma confusão comum entre iniciantes: `print` **mostra** na tela, `return` **devolve** um valor. Uma função que só imprime devolve `None`, e não dá para usar o resultado dela em outro cálculo.

```
def soma_print(a, b):
    print(a + b)

def soma_return(a, b):
    return a + b

x = soma_print(2, 3)
y = soma_return(2, 3)
print("x =", x)
print("y =", y)
```

SAÍDA

```
5
x = None
y = 5
```

6.4 Valores padrão e argumentos nomeados

Um parâmetro pode ter um **valor padrão**, usado quando o argumento não é informado. Ao chamar, podemos também **nomear** o argumento, o que deixa a leitura clara.

```
def aplicar_desconto(valor, percentual=10):
    return valor * (1 - percentual / 100)

print(aplicar_desconto(200))
print(aplicar_desconto(200, 25))
print(aplicar_desconto(200, percentual=50))
```

SAÍDA

```
180.0
150.0
100.0
```

6.5 Escopo: variáveis locais

Variáveis criadas **dentro** da função existem só dentro dela, e desaparecem quando ela termina. É uma proteção: o que acontece lá dentro não bagunça o resto do código. Para mostrar o erro sem interromper o notebook, usamos o `try` e o `except` do Módulo 10 da Parte 1: “tente executar isto; se der erro, faça aquilo”.

```
def calcular():
    resultado_interno = 42
    return resultado_interno

print(calcular())

try:
    print(resultado_interno)
except NameError as erro:
    print("Erro:", erro)
```

SAÍDA

```
42
Erro: name 'resultado_interno' is not defined
```

6.6 Documentando com docstring

Um texto entre aspas triplas logo na primeira linha da função é a **docstring**. Explica o que a função faz, e aparece quando alguém pede ajuda sobre ela com `help()`.

```
def aplicar_desconto(valor, percentual=10):
    """Devolve o valor com o desconto percentual aplicado."""
    return valor * (1 - percentual / 100)

help(aplicar_desconto)
```

SAÍDA

```
Help on function aplicar_desconto in module __main__:

aplicar_desconto(valor, percentual=10)
    Devolve o valor com o desconto percentual aplicado.
```

6.7 Funções que usam decisão

Juntando com o Módulo 2: uma função que classifica uma venda. Note que o `return` encerra a função na primeira condição verdadeira.

```
def classificar_venda(valor):
    if valor >= 1000:
        return "grande"
    elif valor >= 100:
        return "média"
    else:
        return "pequena"

for v in [4500, 250, 89.9]:
    print(v, classificar_venda(v))
```

SAÍDA

```
4500 grande
250 média
89.9 pequena
```

E a função `brl`, do mini projeto da Parte 1, que formata dinheiro no padrão brasileiro. Agora você entende cada linha dela:

```
def brl(valor):
    texto = f"{valor:,.2f}"
    texto = texto.replace(",", "X").replace(".", ",").replace("X", ".")
    return f"R$ {texto}"

print(brl(5200))
print(brl(89.9))
```

SAÍDA

```
R$ 5.200,00
R$ 89,90
```

Na engenharia de dados. Funções pequenas, com um nome claro e uma única responsabilidade, são a base de pipelines que dá para testar e manter. Em vez de um script de 300 linhas, você escreve dez funções de 30.

7. Mini projeto: validação de vendas e relatório por cliente

Vamos juntar tudo num fluxo muito parecido com o de uma pipeline real: **receber dados com problemas, separar o que é válido do que não é, calcular métricas e gerar um relatório**. Cada passo é uma célula. Usaremos as funções `classificar_venda` e `brl` do Módulo 6.

Passo 1: os dados brutos

Desta vez a tabela tem problemas, como os dados reais costumam ter: um cliente vazio (id 4), um valor ausente (id 5) e um valor negativo (id 7).

```
vendas_brutas = [
    {"id": 1, "cliente": "Ana",   "produto": "Notebook", "valor": 4500.00},
    {"id": 2, "cliente": "Bruno", "produto": "Mouse",    "valor": 89.90},
    {"id": 3, "cliente": "Ana",   "produto": "Teclado",  "valor": 250.00},
    {"id": 4, "cliente": "",      "produto": "Monitor",  "valor": 1200.00},
    {"id": 5, "cliente": "Bruno", "produto": "Notebook", "valor": None},
    {"id": 6, "cliente": "Carla", "produto": "Monitor",  "valor": 1200.00},
    {"id": 7, "cliente": "Diego", "produto": "Mouse",    "valor": -79.90},
    {"id": 8, "cliente": "Bruno", "produto": "Notebook", "valor": 5200.00},
]

print(len(vendas_brutas))
```

SAÍDA

```
8
```

Passo 2: uma função que valida um registro

A função devolve o **motivo da rejeição**, ou `None` se o registro estiver bom. Repare na ordem dos testes: checamos o `None` **antes** de comparar o valor com zero, porque comparar `None` com número gera `TypeError`. Como cada `return` encerra a função, não precisamos de `elif`.

```
def motivo_rejeicao(venda):
    """Devolve o motivo da rejeição, ou None se o registro for válido."""
    if not venda["cliente"]:
        return "cliente vazio"
    if venda["valor"] is None:
        return "valor ausente"
    if venda["valor"] <= 0:
        return "valor não positivo"
    return None
```

Testando em três registros: o primeiro é bom, o quarto tem cliente vazio e o quinto tem valor ausente.

```
print(motivo_rejeicao(vendas_brutas[0]))
print(motivo_rejeicao(vendas_brutas[3]))
print(motivo_rejeicao(vendas_brutas[4]))
```

SAÍDA

```
None
cliente vazio
valor ausente
```

Passo 3: separando válidos e rejeitados

Um `for` percorre a tabela e um `if` decide o destino de cada registro. Os rejeitados guardamos como **tupla** (id, motivo), para depois reportar.

```
validas = []
rejeitadas = []

for venda in vendas_brutas:
    motivo = motivo_rejeicao(venda)
    if motivo is None:
        validas.append(venda)
    else:
        rejeitadas.append((venda["id"], motivo))

print("Válidas:", len(validas))
print("Rejeitadas:", rejeitadas)
```

SAÍDA

```
Válidas: 5
Rejeitadas: [(4, 'cliente vazio'), (5, 'valor ausente'), (7, 'valor não positivo')]
```

Passo 4: classificando as vendas válidas

Uma comprehension aplica a função `classificar_venda` a cada venda válida. O método `.count()` de uma lista conta quantas vezes um valor aparece nela.

```
faixas = [classificar_venda(v["valor"]) for v in validas]

print(faixas)
print("grande: ", faixas.count("grande"))
print("média:  ", faixas.count("média"))
print("pequena:", faixas.count("pequena"))
```

SAÍDA

```
['grande', 'pequena', 'média', 'grande', 'grande']
grande:  3
média:   1
pequena: 1
```

Passo 5: total por cliente, em uma única passada

Este é o problema do Módulo 3.8, resolvido do jeito eficiente: um **dicionário acumulador**. A chave é o cliente e o valor é o total acumulado. Para cada venda, se o cliente já está no dicionário, somamos; senão, criamos a entrada.

```
totais = {}

for venda in validas:
    cliente = venda["cliente"]
    if cliente in totais:
        totais[cliente] += venda["valor"]
    else:
        totais[cliente] = venda["valor"]

print(totais)
```

SAÍDA

```
{'Ana': 4750.0, 'Bruno': 5289.9, 'Carla': 1200.0}
```

O método `.get(chave, padrão)` da Parte 1 permite encurtar o `if` e o `else` numa linha só: se o cliente ainda não existe, o `.get` devolve 0 e somamos a partir dele.

```
totais_v2 = {}

for venda in validas:
    cliente = venda["cliente"]
    totais_v2[cliente] = totais_v2.get(cliente, 0) + venda["valor"]

print(totais_v2 == totais)
```

SAÍDA

```
True
```

Passo 6: ranking dos clientes

Para ordenar do maior para o menor total, usamos `sorted()`. O método `.items()` entrega pares (cliente, total), e o parâmetro `key` recebe **uma função que diz por qual parte ordenar**. Criamos uma função que devolve a posição 1 do par, que é o total. O `reverse=True` inverte a ordem.

```
def pegar_total(par):
    return par[1]

ranking = sorted(totais.items(), key=pegar_total, reverse=True)

print(ranking)
```

SAÍDA

```
[('Bruno', 5289.9), ('Ana', 4750.0), ('Carla', 1200.0)]
```

Passo 7: o relatório

Usamos o `enumerate` para numerar o ranking e a função `brl` para formatar os valores. Dois detalhes novos nas f-strings: `{cliente:<8}` reserva 8 espaços e alinha o texto à **esquerda**, e `{...:>14}` reserva 14 e alinha à **direita**, para os valores formarem uma coluna.

```
print("=" * 40)
print("RANKING DE CLIENTES")
print("=" * 40)

for posicao, par in enumerate(ranking, start=1):
    cliente, total = par
    print(f"{posicao}. {cliente:<8} {brl(total):>14}")

print("=" * 40)
print(f"Registros válidos:    {len(validas)}")
print(f"Registros rejeitados: {len(rejeitadas)}")
for id_registro, motivo in rejeitadas:
    print(f"   id {id_registro}: {motivo}")
print(f"Vendas grandes:       {faixas.count('grande')}")
```

SAÍDA

```
========================================
RANKING DE CLIENTES
========================================
1. Bruno       R$ 5.289,90
2. Ana         R$ 4.750,00
3. Carla       R$ 1.200,00
========================================
Registros válidos:    5
Registros rejeitados: 3
   id 4: cliente vazio
   id 5: valor ausente
   id 7: valor não positivo
Vendas grandes:       3
```

Em poucas dezenas de linhas você validou dados, classificou, agregou, ordenou e gerou um relatório legível. É o esqueleto de qualquer processamento de dados, e é exatamente o que ferramentas como pandas e Spark fazem em escala.

8. Erros comuns em decisões, laços e funções

Erro ou sintoma	Causa mais comum	Exemplo
`IndentationError`	Recuo faltando ou inconsistente depois de `:`	`for x in lista:` seguido de linha sem recuo
`SyntaxError`	Dois-pontos esquecido no fim do `if`, `for`, `while` ou `def`	`if x > 5`
`SyntaxError`	Atribuição `=` no lugar da comparação `==`	`if x = 5:`
Laço que não termina	A condição do `while` nunca fica falsa	`while x > 0:` sem alterar `x`
`TypeError`	Comparar `None` com número	`None > 0`
`NameError`	Usar fora da função uma variável criada dentro dela	`print(resultado_interno)`
Resultado `None`	Função sem `return`, que só imprime	`x = soma_print(2, 3)`
Uma volta a menos	O fim do `range` não é incluído	`range(1, 5)` vai só até 4
Classificação errada	Ordem das condições do `if` e `elif`	Testar `>= 100` antes de `>= 1000`

Dica. Quando um laço ou um `if` não faz o que você espera, coloque um `print` das variáveis logo dentro dele. Ver o valor a cada volta resolve a maioria dos mistérios.

9. Exercícios

Exercício 1: Decisão com elif. Dada a variável `nota = 6.5`, imprima `Aprovado` se a nota for 7 ou mais, `Recuperação` se for 5 ou mais (e menos de 7), e `Reprovado` nos demais casos. Teste também com 8 e com 3.

Exercício 2: Laço com acumulador. Dada a lista `valores = [50, 120, 300, 80, 150]`, use um `for` para calcular o total e depois imprima o total e a média (total dividido pela quantidade).

Exercício 3: Enumerate. Dada a lista `produtos = ["Mouse", "Teclado", "Monitor"]`, imprima cada produto numerado a partir de 1, no formato `1. Mouse`.

Exercício 4: Laço while. Comece com `saldo = 1000` e, usando um `while`, dobre o saldo a cada volta até ele passar de 10000. Imprima quantas voltas foram necessárias e o saldo final.

Exercício 5: List comprehension. Dada a lista `precos = [19.9, 45.0, 8.5, 120.0, 63.2]`, crie com uma comprehension a lista dos preços maiores que 20, já com 10% de desconto e arredondados em 2 casas.

Exercício 6: Função. Crie a função `ticket_medio(valores)` que devolve a média arredondada em 2 casas, ou `0` se a lista estiver vazia. Teste com `[50, 120, 300]` e com `[]`.

Exercício 7: Dicionário acumulador. Dada a lista `compras = ["mouse", "teclado", "mouse", "monitor", "mouse", "teclado"]`, use um `for` e um dicionário para contar quantas vezes cada produto aparece.

10. Resumo e próximos passos

Conceito	Em uma frase
`if`, `elif`, `else`	Executam um bloco só se a condição for verdadeira. O Python para na primeira que for
Condições compostas	`and`, `or`, `not` e `in` combinam condições
`for`	Repete o bloco para cada item de uma coleção
`range`	Fabrica uma sequência de números inteiros (o fim não entra)
`enumerate`	Entrega a posição junto com o item
`while`	Repete enquanto a condição for verdadeira
`break` e `continue`	Sair do laço, ou pular para a próxima volta
Acumulador	Variável criada antes do laço e atualizada a cada volta
Comprehension	Atalho de uma linha para montar uma lista, set ou dict
Função	Bloco de código com nome, parâmetros e `return`
Escopo	Variável criada dentro da função só existe nela

Gabarito dos exercícios

Existe mais de um jeito correto de resolver cada exercício. Tente primeiro sozinho e compare depois.

Exercício 1: Decisão com elif

```
nota = 6.5

if nota >= 7:
    print("Aprovado")
elif nota >= 5:
    print("Recuperação")
else:
    print("Reprovado")
```

SAÍDA

```
Recuperação
```

Exercício 2: Laço com acumulador

```
valores = [50, 120, 300, 80, 150]

total = 0
for v in valores:
    total += v

print("Total:", total)
print("Média:", total / len(valores))
```

SAÍDA

```
Total: 700
Média: 140.0
```

Exercício 3: Enumerate

```
produtos = ["Mouse", "Teclado", "Monitor"]

for posicao, produto in enumerate(produtos, start=1):
    print(f"{posicao}. {produto}")
```

SAÍDA

```
1. Mouse
2. Teclado
3. Monitor
```

Exercício 4: Laço while

```
saldo = 1000
voltas = 0

while saldo <= 10000:
    saldo = saldo * 2
    voltas += 1

print(voltas, saldo)
```

SAÍDA

```
4 16000
```

Exercício 5: List comprehension

```
precos = [19.9, 45.0, 8.5, 120.0, 63.2]

com_desconto = [round(p * 0.9, 2) for p in precos if p > 20]

print(com_desconto)
```

SAÍDA

```
[40.5, 108.0, 56.88]
```

Exercício 6: Função

```
def ticket_medio(valores):
    if not valores:
        return 0
    return round(sum(valores) / len(valores), 2)

print(ticket_medio([50, 120, 300]))
print(ticket_medio([]))
```

SAÍDA

```
156.67
0
```

Exercício 7: Dicionário acumulador

```
compras = ["mouse", "teclado", "mouse", "monitor", "mouse", "teclado"]

contagem = {}
for item in compras:
    contagem[item] = contagem.get(item, 0) + 1

print(contagem)
```

SAÍDA

```
{'mouse': 3, 'teclado': 2, 'monitor': 1}
```

Glossário

Termo	Significado
Condição	Expressão que resulta em True ou False
Decisão (condicional)	Escolha de um caminho do código conforme uma condição
Laço (loop)	Bloco de código que o Python repete várias vezes
Iterar	Percorrer os itens de uma coleção, um de cada vez
Volta (iteração)	Cada repetição do corpo de um laço
Acumulador	Variável criada antes do laço e atualizada a cada volta (soma, contagem)
Laço infinito	Laço cuja condição nunca fica falsa, e que nunca termina
Comprehension	Forma curta de montar uma lista, set ou dict a partir de um laço
Expressão condicional	Escolha entre dois valores em uma linha: `a if condição else b`
Função	Bloco de código com nome, que executa uma tarefa quando chamado
Parâmetro	Nome da entrada de uma função, na sua definição
Argumento	Valor passado a uma função quando ela é chamada
Retorno (return)	Valor que a função devolve a quem a chamou
Escopo	Região do código em que uma variável existe
Docstring	Texto entre aspas triplas que documenta uma função
Fila	Lista de itens a processar, um após o outro
Retry	Nova tentativa de uma operação que falhou